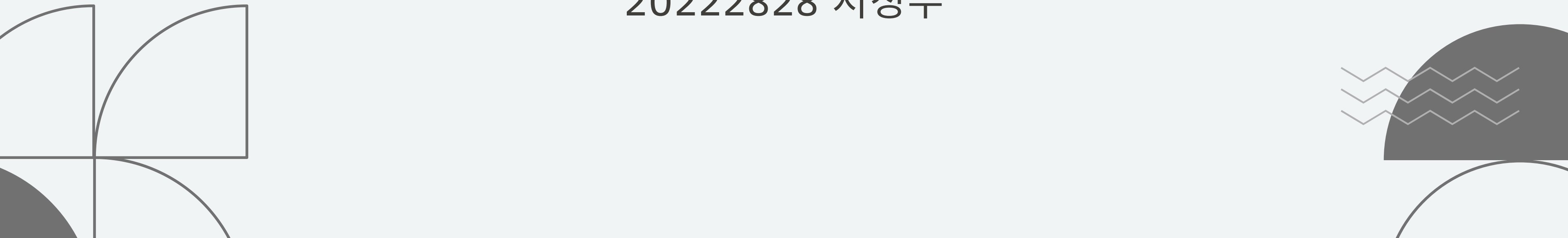
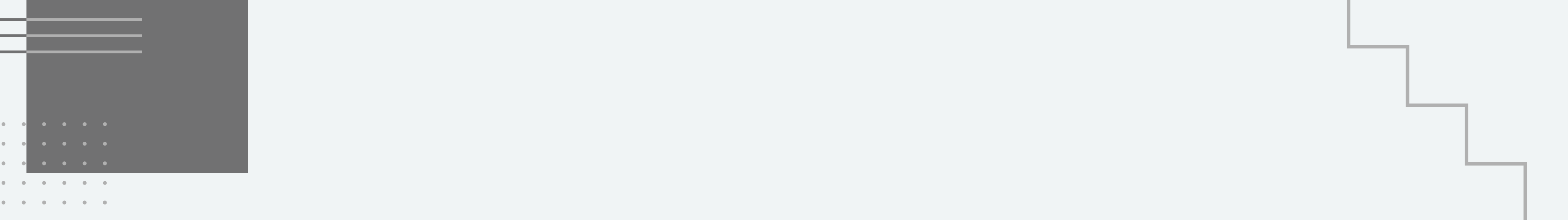


머신러닝 기반 음식 배달 지연 예측

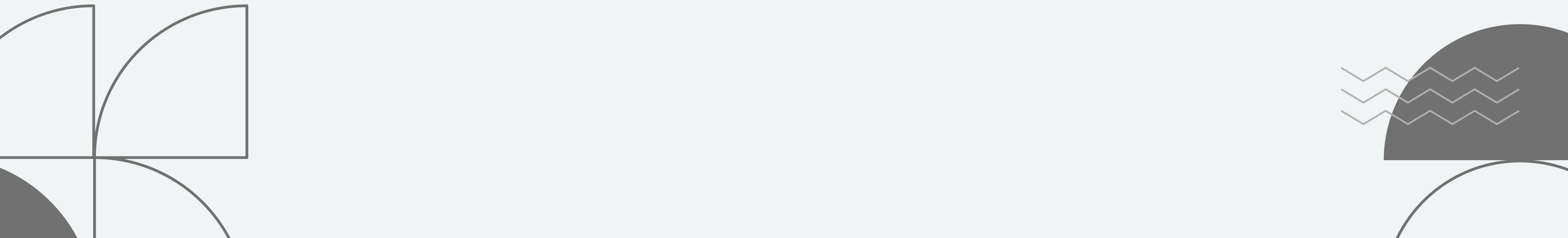
20211720 강민혁

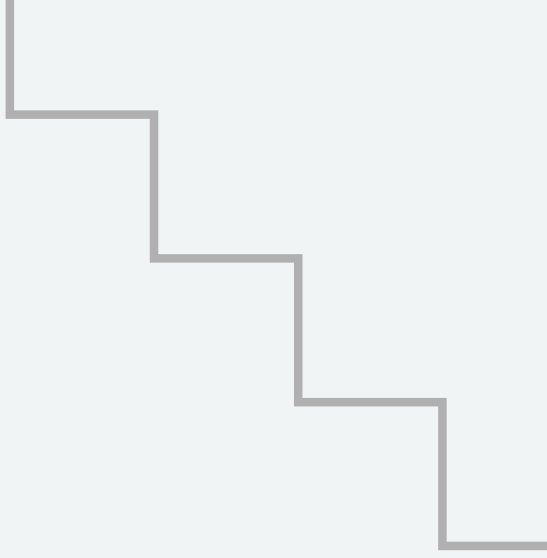
20222828 서정우





CONTENTS

- 문제 정의
 - 데이터셋 개요
 - EDA & 핵심 지표
 - 데이터 전처리
 - 모델 훈련
 - 분석 및 결과
 - 특성 중요도
- 

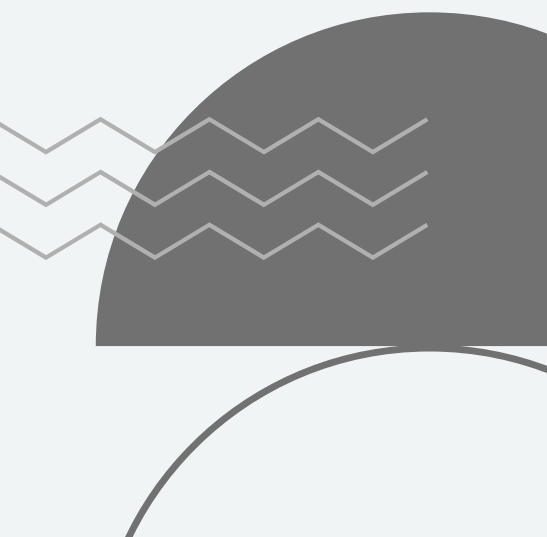
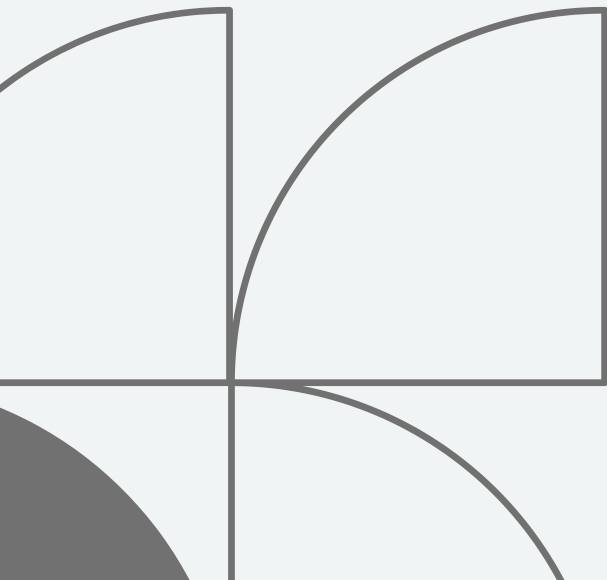


PROBLEM DEFINITION

배달 지연은 고객 관리에 영향을 미치는 운영 리스크입니다.

프로젝트 목표

주문 정보를 바탕으로 배달 지연 가능성을 예측하는 모델 구축



DATASET OVERVIEW

15000

총 주문건수

30

원본변수

9.5%

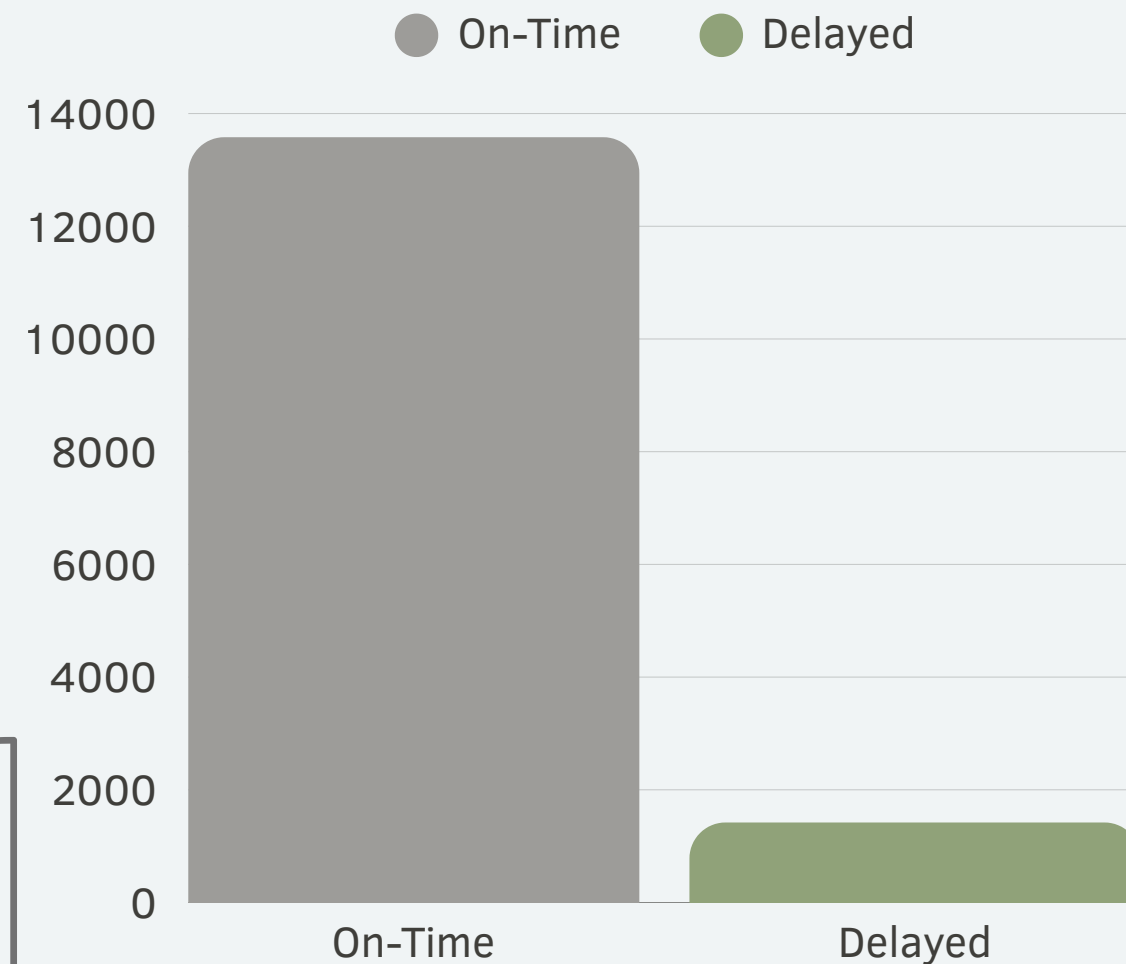
자연 배달 비율

이진분류

결과 예측

도시 음식 배달 운영을 현실적으로 모사한 합성 데이터셋으로,
배달 시간.교통.날씨.고객 행동.주문 금액.평점.취소/환불 등 주요
지표를 담아 분석과 예측 모델링에 활용할 수 있습니다.

EDA & KEY FINDINGS



1. 데이터 불균형

정시 90.5% VS 지연 9.5%

2. 단일변수의 한계

교통·거리·조리 시간의 그룹 간 분포 유사,
파생 변수 설계 필요

PREPROCESSING

ID 열 제거

결측치 보정

인코딩

BOOLEAN → INT

학습 테스트

8:2 분할

FEATURE ENGINEERING

STRESS_SCORE

IS_PEAK_HOUR

DELIVERY_SPEED

```
# Feature 1: stress score
# High traffic + bad weather = more stress on delivery
df['stress_score'] = df['traffic_level_score'] *
df['weather_severity_score']

# Feature 2: Is it a peak hour?
df['is_peak_hour'] = df['order_hour'].between(11, 14) |
df['order_hour'].between(18, 22)
df['is_peak_hour'] = df['is_peak_hour'].astype(int)

# Feature 3: Speed of partner (km per minute)
df['delivery_speed'] = df['delivery_distance_km'] /
(df['delivery_time_minutes'] + 1)
```

MODEL TRAINING

모델 선정

1. KNN

2. XGBOOST

MODEL TRAINING

모델의 학습 방법

1.KNN

특정 공간에 분포하는 데이터에 대하여
K개의 가장 가까운 이웃을 살펴보고 다수
결 방식으로 데이터의 레이블을 할당하는
분류 방식

MODEL TRAINING

모델의 학습 방법

2.XGBOOST

의사결정나무를 활용하는 알고리즘으로 처음엔 단순한 기준으로 시작하고, 앞선 예측의 오차를 줄여 나가는 방식으로 학습하며 나무를 키우는 분류 방식

MODEL TRAINING

모델의 구현

1. KNN

```
# KNN 모델 학습
print("\n[KNN 모델 학습 시작]")
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_sc, y_train)
print("\n[KNN 모델 학습 완료]")
```

MODEL TRAINING

모델의 구현

2.XGBOOST

```
# XGBoost 모델 학습
weight = y_train.value_counts()[0] / y_train.value_counts()[1]

print("\n[XGBoost 모델 학습 시작]")
xgb = XGBClassifier(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=5,
    random_state=42,
    scale_pos_weight=weight,
    use_label_encoder=False,
    eval_metric='logloss'
)
xgb.fit(X_train, y_train)
print("\n[XGBoost 모델 학습 완료]")
```

MODEL EVALUATION

```
# 모델 평가 함수 정의
def evaluate_model (name, model, X_test_data, y_test_data):
    y_pred = model.predict(X_test_data)
    y_pred_prob = model.predict_proba(X_test_data)[:, 1]

    acc = accuracy_score(y_test_data, y_pred)
    auc = roc_auc_score(y_test_data, y_pred_prob)

    print(f'\n{'='*50}')
    print(f'🚩 {name}')
    print(f'{'='*50}')
    print(f'  Accuracy : {acc:.4f}  ({acc*100:.2f}%)')
    print(f'  AUC-ROC   : {auc:.4f}')
    print()
    print(classification_report(y_test_data, y_pred, target_names=['On Time', 'Delayed']))

    return auc, y_pred_prob

auc_knn, prob_knn = evaluate_model('KNN', knn, X_test_sc, y_test)
auc_xgb, prob_xgb = evaluate_model('XGBoost', xgb, X_test, y_test)
```


MODEL EVALUATION

```
=====
📌 KNN
=====
Accuracy : 0.9013 (90.13%)
AUC-ROC  : 0.5391

      precision    recall  f1-score   support

 On Time         0.91      0.99      0.95     2716
 Delayed         0.12      0.01      0.01       284

 accuracy
macro avg         0.52      0.50      0.48     3000
weighted avg         0.83      0.90      0.86     3000

=====
📌 XGBoost
=====
Accuracy : 0.9777 (97.77%)
AUC-ROC  : 0.9972

      precision    recall  f1-score   support

 On Time         1.00      0.98      0.99     2716
 Delayed         0.82      0.98      0.89       284

 accuracy
macro avg         0.91      0.98      0.94     3000
weighted avg         0.98      0.98      0.98     3000
=====
```

ACCURACY : 정확도

모델의 예측 중 정답의 비율, 데이터의 불균형을 반영하지 못 함

AUC-ROC

진양성(TP)과 위양성(FP) 사이의 관계

MODEL EVALUATION

=====				
📌 KNN				
=====				
Accuracy : 0.9013 (90.13%)				
AUC-ROC : 0.5391				
	precision	recall	f1-score	support
On Time	0.91	0.99	0.95	2716
Delayed	0.12	0.01	0.01	284
accuracy			0.90	3000
macro avg	0.52	0.50	0.48	3000
weighted avg	0.83	0.90	0.86	3000
=====				
📌 XGBoost				
=====				
Accuracy : 0.9777 (97.77%)				
AUC-ROC : 0.9972				
	precision	recall	f1-score	support
On Time	1.00	0.98	0.99	2716
Delayed	0.82	0.98	0.89	284
accuracy			0.98	3000
macro avg	0.91	0.98	0.94	3000
weighted avg	0.98	0.98	0.98	3000

PRECISION : 정밀도

모델이 참이라고 분류 했을 때 실제로 참인 비율

RECALL : 재현율

실제 참인 것을 모델이 참으로 분류한 비율

F1 SCORE : F1 점수

정밀도와 재현율의 조화평균

SUPPORT

모델의 예측을 채점할 때 사용한 정답 데이터의 개수

MODEL EVALUATION

```
=====
📌 KNN
=====
Accuracy : 0.9013 (90.13%)
AUC-ROC  : 0.5391

      precision    recall  f1-score   support

 On Time         0.91      0.99      0.95     2716
 Delayed         0.12      0.01      0.01       284

 accuracy
macro avg         0.52      0.50      0.48     3000
weighted avg         0.83      0.90      0.86     3000

=====
📌 XGBoost
=====
Accuracy : 0.9777 (97.77%)
AUC-ROC  : 0.9972

      precision    recall  f1-score   support

 On Time         1.00      0.98      0.99     2716
 Delayed         0.82      0.98      0.89       284

 accuracy
macro avg         0.91      0.98      0.94     3000
weighted avg         0.98      0.98      0.98     3000
=====
```

MACRO AVG

단순히 점수를 더하고 둘로 나눈 평균

WEIGHTED AVG

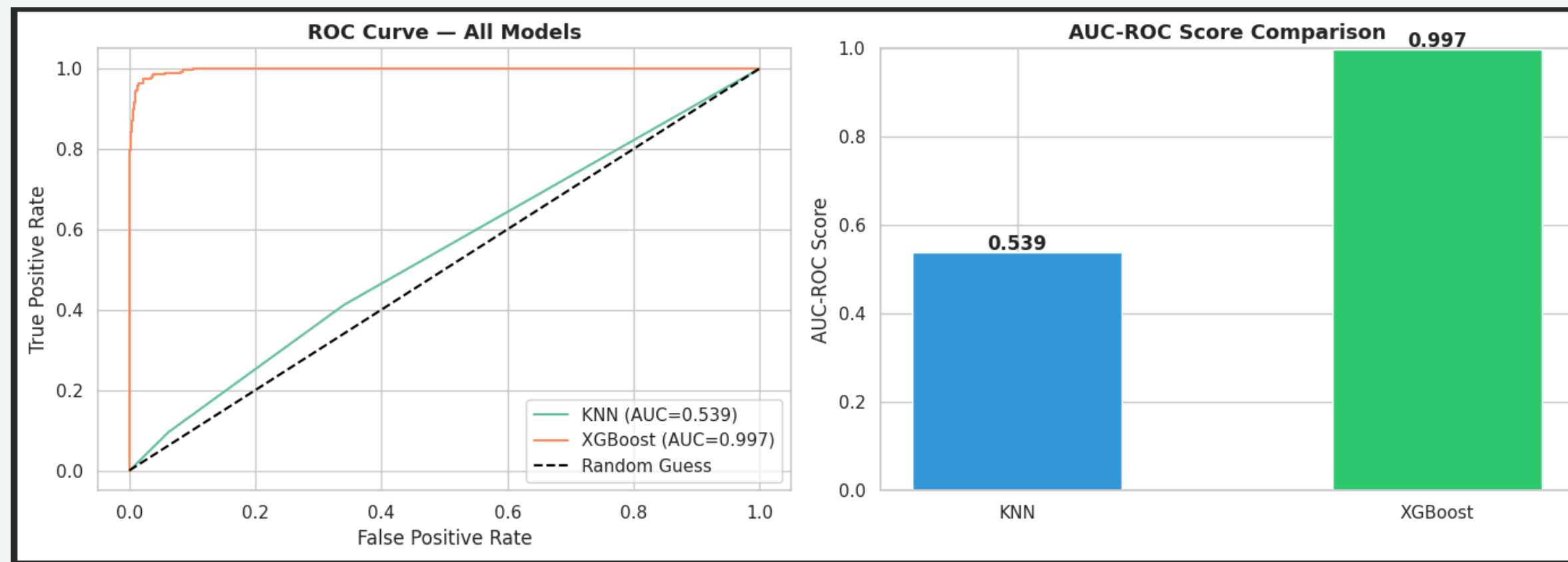
데이터 개수가 더 많은 쪽에 가중치를 두고 계산한 평균

ANALYSIS

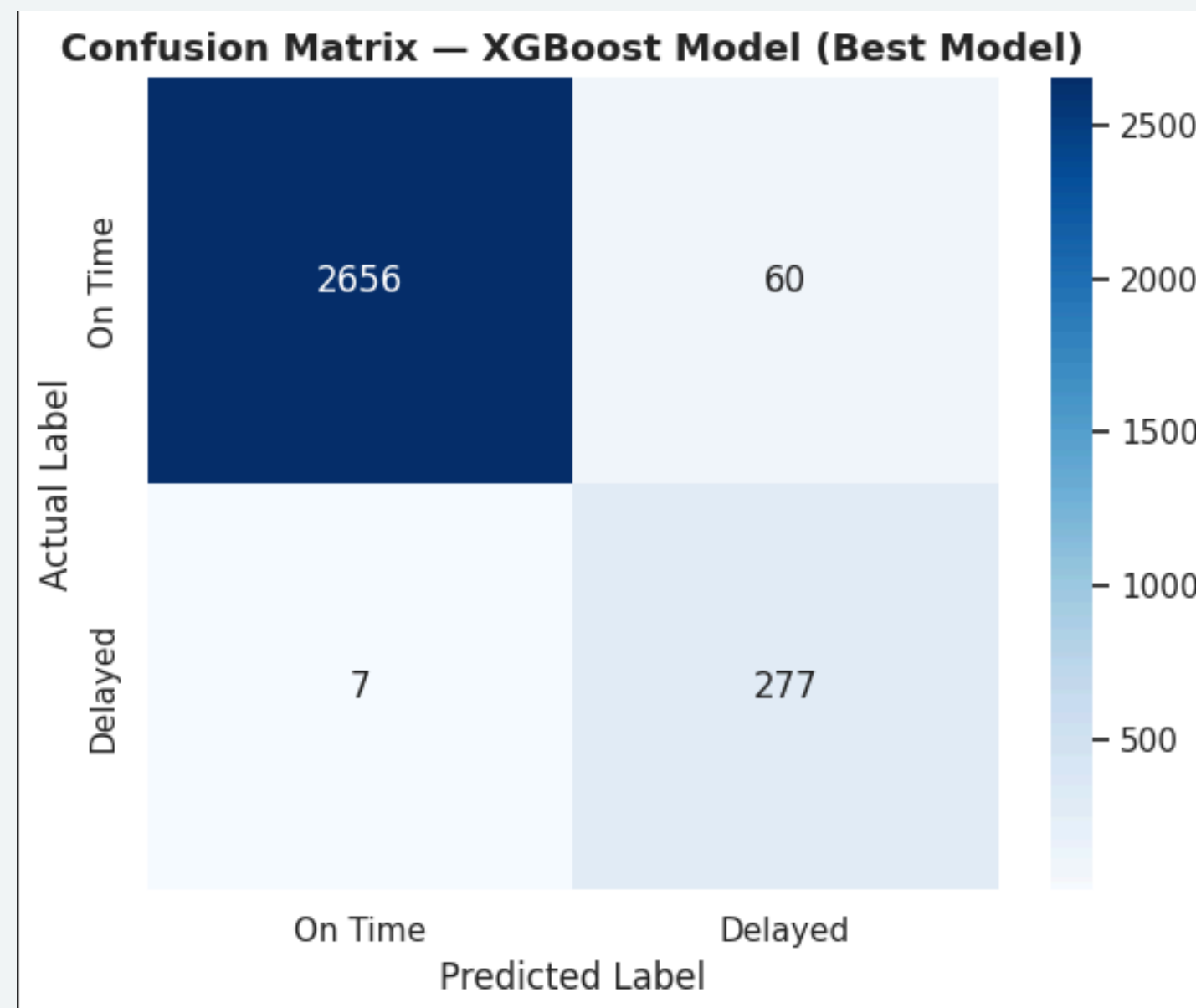
=====				
📌 KNN				
=====				
Accuracy : 0.9013 (90.13%)				
AUC-ROC : 0.5391				
	precision	recall	f1-score	support
On Time	0.91	0.99	0.95	2716
Delayed	0.12	0.01	0.01	284
accuracy			0.90	3000
macro avg	0.52	0.50	0.48	3000
weighted avg	0.83	0.90	0.86	3000
=====				
📌 XGBoost				
=====				
Accuracy : 0.9777 (97.77%)				
AUC-ROC : 0.9972				
	precision	recall	f1-score	support
On Time	1.00	0.98	0.99	2716
Delayed	0.82	0.98	0.89	284
accuracy			0.98	3000
macro avg	0.91	0.98	0.94	3000
weighted avg	0.98	0.98	0.98	3000

- 1.KNN은 정확도가 높아 좋은 모델 같지만, AUC-ROC 점수를 보면 그렇지 않다는 것을 알 수 있습니다.
- 2.KNN은 지연 배달 데이터에 대한 점수가 떨어지는 것을 확인할 수 있습니다.
- 3.반면에 XGBOOST는 정확도와 AUC-ROC 점수가 모두 높게 나온 것을 알 수 있습니다.
- 4.XGBOOST는 데이터 불균형에도 불구하고 정상 배달과 지연 배달에 대한 정밀도와 재현율이 모두 높게 나온 것을 알 수 있습니다.

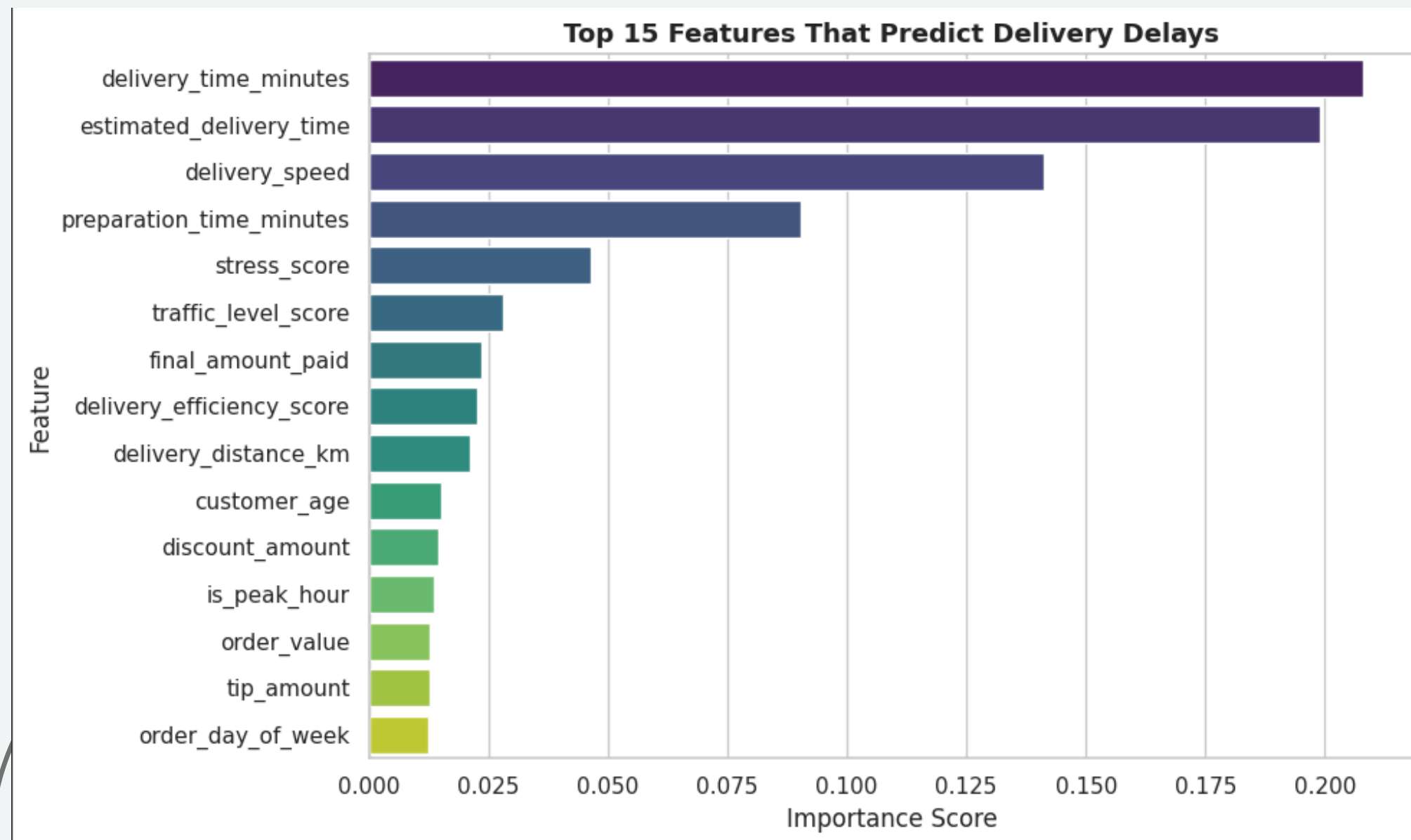
MODEL EVALUATION



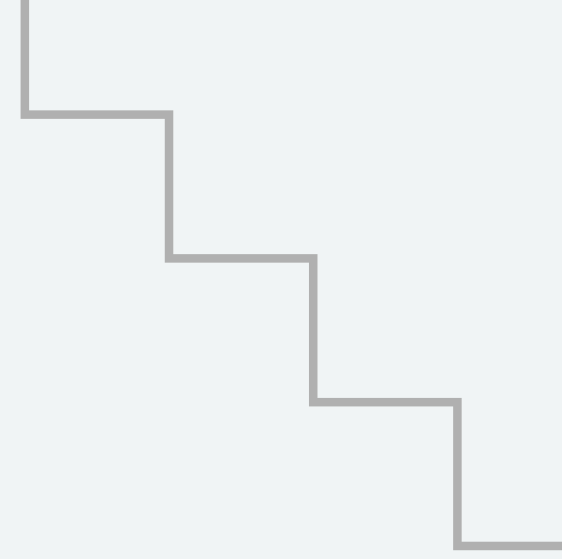
RESULT



FEATURE IMPORTANCE



1. 배달 시간
2. 추정된 배달 시간
3. 배달 속도
4. 스트레스 점수
5. 교통 점수



**THANK
YOU**

